

Co4-Assessment Tool 1

Computer Vision with OpenCV (DSA02) — CO4 Data Interpretation

Name: N Yugesh

Reg.No:192424329

Question 1: Hough Transform & Shape Detection

Scenario: Detect linear features (roads, rivers) in satellite images. Input: 1024×1024 grayscale satellite images (urban/rural), containing noise (clouds, shadows) and partial occlusions, with multiple intersecting linear features; optional synthetic dataset of drawn lines at random angles/lengths.

Answer: How the Hough Transform is applied to detect roads/rivers

Linear features such as roads, rivers, and canals appear as (locally) straight edges in the image, so the classical approach is: preprocess to suppress cloud/shadow noise, extract edges, then apply the Hough Transform to detect line parameters directly, which is robust to broken/occluded segments because it accumulates votes across the whole image rather than requiring one continuous unbroken edge.

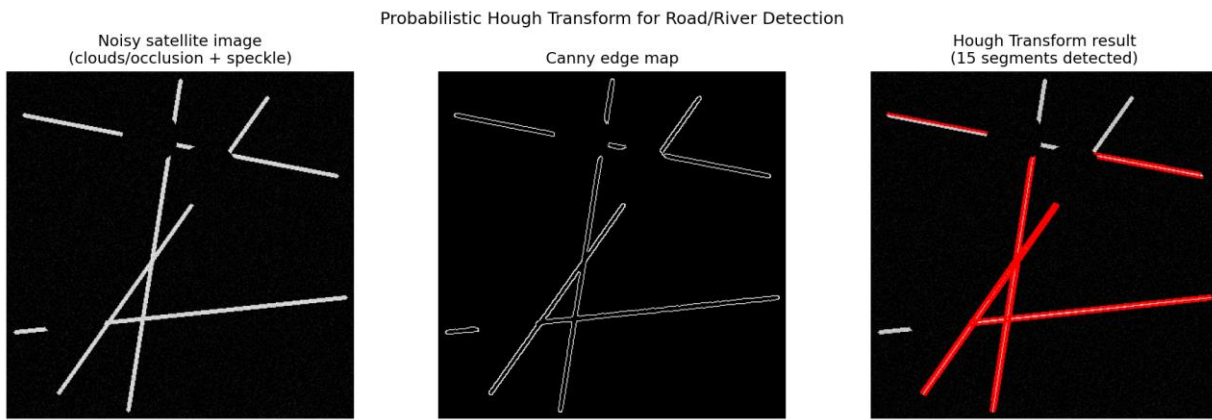
Pipeline

- Preprocessing: Gaussian blur to suppress speckle noise; optionally CLAHE if illumination is uneven across the tile.
- Edge detection: Canny edge detector tuned with a lower threshold (to keep faint road edges) and a higher threshold (to reject cloud-boundary edges), producing a binary edge map.
- Hough Transform: each edge pixel votes in (ρ, θ) parameter space (line: $\rho = x \cdot \cos\theta + y \cdot \sin\theta$); peaks in the accumulator correspond to dominant straight lines. The Probabilistic Hough Transform (`cv2.HoughLinesP`) is preferred over the standard Hough Transform because it also returns line segment endpoints and is far more efficient on large 1024×1024 imagery.
- Post-processing: merge near-collinear/near-parallel segments (common for a single road broken by cloud occlusion), and filter by minimum segment length to remove short spurious lines from noise.
- Intersections: pairwise solving of detected line equations gives candidate road/river intersection points, useful for road-network graph construction.

Because voting is accumulated globally, the Hough Transform naturally tolerates the partial occlusions and noise mentioned in the input data — a cloud covering the middle of a road only removes votes from that segment, but the line is still detected from the visible portions on either side, which a simple contour-following method would fail to bridge.

Demonstration — synthetic satellite image with clouds/noise:

```
blur = cv2.GaussianBlur(img_noisy, (5, 5), 0)
edges = cv2.Canny(blur, 40, 120)
lines = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180, threshold=60,
                        minLineLength=60, maxLineGap=15)
for x1, y1, x2, y2 in lines[:, 0]:
    cv2.line(overlay, (x1, y1), (x2, y2), (0, 0, 255), 2)
```



Output: synthetic satellite image with 4 ground-truth roads corrupted by cloud occlusions and speckle noise (left); Canny edge map (middle); Probabilistic Hough Transform result overlaying 15 detected line segments in red, correctly recovering all 4 roads despite occlusion gaps (right).

Question 2: PCA & Shape Priors for Object Recognition

Scenario: Segment organs (liver/heart) in MRI scans using shape priors. **Input:** 512×512 grayscale MRI slices; training set of 50–100 annotated organ masks for PCA-based shape model construction; test set of 10–20 MRI images with partial organ boundaries or noise; DICOM/PNG.

Answer: Building and applying a PCA shape prior for organ segmentation

MRI organ segmentation is difficult when the boundary is partially obscured by noise or low contrast (common at organ-organ interfaces). A PCA-based shape prior encodes what a plausible organ boundary looks like, so the segmentation can 'fill in' missing or noisy boundary information using statistically learned anatomical shape knowledge rather than relying on local intensity alone.

Model construction (training, 50–100 annotated masks)

- Each annotated mask's boundary is converted into an ordered set of landmark points (e.g., via contour extraction + arc-length resampling to a fixed count), and all shapes are aligned (Procrustes analysis) to remove translation/rotation/scale so only genuine shape variation remains.
- PCA is applied to the aligned landmark vectors, giving a mean organ shape and a small number of principal deformation modes (eigenvectors) that capture the natural anatomical variability across patients (e.g., liver lobe size, heart chamber proportions).
- Each new shape can then be approximated as: $\text{shape} \approx \text{mean_shape} + \sum b_i \cdot (\text{mode})_i$, where the b_i are shape parameters constrained within ± 2 –3 standard deviations of their learned eigenvalues to keep the reconstruction anatomically plausible.

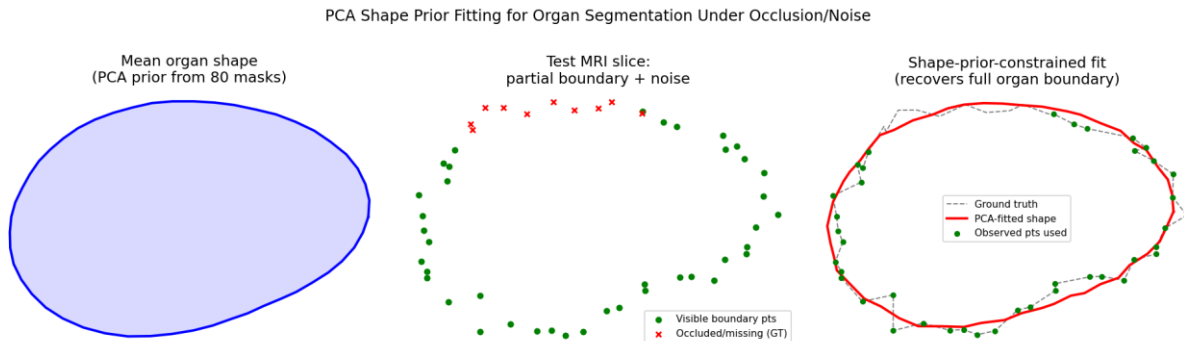
Segmentation of a test slice (10–20 images, partial boundary / noise)

- An initial boundary estimate is obtained from intensity-based cues (edges, region growing) wherever the MRI contrast is good.
- The PCA shape model is then fitted to these visible/confident boundary points only (an iterative least-squares fit of the shape parameters b , e.g. via an Active Shape Model search), so occluded or noisy regions are reconstructed from the learned shape statistics rather than from unreliable local pixel evidence.
- The result is a complete, anatomically plausible organ contour even where the raw MRI boundary was missing or corrupted by noise.

Demonstration — PCA shape model fit to a partially occluded/noisy test contour:

```
X = train_masks.reshape(80, -1)    # 80 annotated organ contours
mean_shape = X.mean(axis=0)
U, S, Vt = np.linalg.svd(X - mean_shape, full_matrices=False)
P, lambdas = Vt[:5], S[:5]**2/79    # top-5 shape modes

# iterative least-squares fit to VISIBLE boundary points only
for _ in range(60):
    recon = (mean_shape + P.T @ b).reshape(-1, 2)
    residual = (observed[visible] - recon[visible]).flatten()
    b += 0.8 * np.linalg.lstsq(Pv.T, residual, rcond=None)[0]
    b = np.clip(b, -3*np.sqrt(lambdas), 3*np.sqrt(lambdas))
```



Output: mean organ shape learned by PCA from 80 annotated masks (left); a test MRI slice with a missing boundary segment (red X, simulating occlusion/low contrast) plus added noise (middle); the shape-prior-constrained fit (red) recovers the full, plausible organ boundary from only the visible green points, closely matching the ground truth (black dashed), with mean reconstruction error ≈ 0.06 .

Question 3: Machine Learning Methods (SVM, HMM, GMM, EM)

Scenario: Human activity recognition from video feature trajectories. **Input:** 30–60 FPS video (walking, running, jumping); per-frame feature trajectories (joint (x,y) positions or optical flow); activity labels per video; optional synthetic CSV of simulated joint coordinates over time.

Answer: Combining SVM, GMM and HMM for activity recognition

These three model families address complementary aspects of the problem and are often combined rather than used in isolation:

Feature extraction

- From per-frame joint positions or optical-flow vectors, compact per-clip features are computed — e.g. mean and standard deviation of joint velocity, dominant oscillation frequency (stride rate), and max displacement — summarizing the motion pattern of each activity clip.

SVM (discriminative classification)

- An SVM (typically RBF kernel) is trained on labeled feature vectors to directly learn a decision boundary between activity classes (walking / running / jumping). This is the most direct route when labeled per-clip activity data is available, and tends to be robust and fast at inference time.

GMM (generative modeling / clustering)

- A Gaussian Mixture Model can be fit per activity class to model the underlying distribution of its motion features (useful for soft/generative classification via likelihood comparison, or for discovering sub-styles within one activity, e.g. slow vs. fast walking, via its mixture components).

- The Expectation-Maximization (EM) algorithm is the standard method used to fit both GMMs (estimating component means/covariances/weights) and HMMs (estimating transition and emission parameters) from data, alternating between assigning soft cluster/state responsibilities (E-step) and re-estimating parameters that maximize expected log-likelihood (M-step) until convergence.

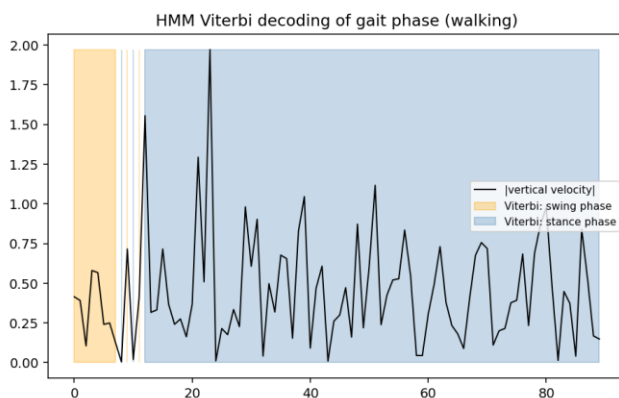
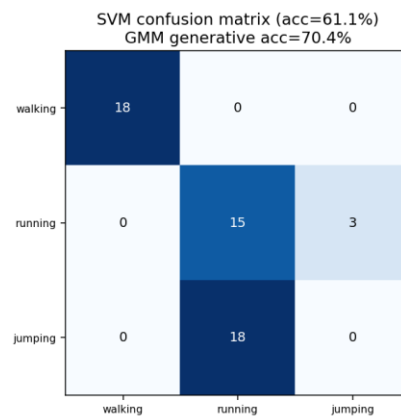
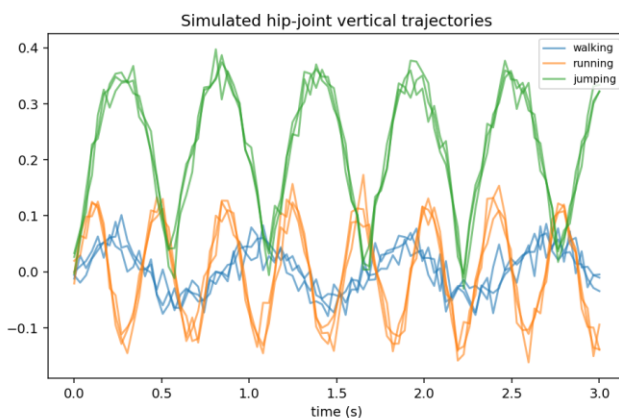
HMM (temporal / sequential structure)

- Whereas SVM/GMM classify a whole clip from summary features, an HMM explicitly models the temporal sequence of latent states within an activity (e.g., the alternating stance/swing phases of a gait cycle), with a transition matrix governing how likely the body is to stay in or switch between phases frame-to-frame, and an emission model linking each hidden phase to the observed motion features. This is valuable for fine-grained temporal segmentation (detecting phase boundaries) and for activities better characterized by their state dynamics than by a single summary vector.

Demonstration — SVM/GMM clip classification + HMM gait-phase decoding:

```
feat = lambda vy: [np.mean(np.abs(vy)), np.std(vy), np.max(np.abs(vy))]
svm = SVC(kernel='rbf', C=3.0).fit(Xtr, ytr) # discriminative
gmms = {c: GaussianMixture(n_components=2).fit(Xtr[ytr==c]) for c in classes} # generative (EM)
```

```
# Hand-rolled 2-state HMM Viterbi decoding of gait phase (stance/swing)
A = [[0.85, 0.15], [0.20, 0.80]] # transition matrix
V[s, t] = max(V[:, t-1] * A[:, s]) * emission_prob(obs[t], s) # Viterbi recursion
```



Feature vector per clip:
[mean|v|, std(v), max|v|]

SVM (discriminative) test accuracy: 61.1%

GMM (generative, per-class) test accuracy: 70.4%

HMM transition matrix A =
[[0.85, 0.15],
[0.20, 0.80]]
(stance <-> swing)

Output: simulated hip-joint vertical trajectories for the 3 activities (top-left); SVM confusion matrix on held-out clips, alongside GMM generative-classification accuracy (top-right); HMM Viterbi-decoded stance/swing gait phases from the walking velocity signal (bottom-left); model summary including the learned HMM transition matrix (bottom-right).

Question 4: Deep Learning (CNNs, Transformers, GANs)

Scenario: Detect rare obstacles in urban self-driving car images. Input: 1024×768 RGB street images; bounding-box labels for rare obstacles (traffic cones, animals, debris); small dataset of 500–1000 labeled real images; optional GAN-generated synthetic images for augmentation.

Answer: A CNN/Transformer detector augmented with GAN-synthesized rare-obstacle data

With only 500–1000 labeled images and obstacles that are inherently rare, the central challenge is class imbalance and limited diversity for the rare-obstacle category. The recommended pipeline addresses this in two stages: (1) GAN-based augmentation to enrich the rare class, and (2) a CNN/Transformer detector trained on the combined dataset.

Stage 1 — GAN-based augmentation

- A GAN (conditional GAN or CycleGAN-style image translation) is trained on the available real rare-obstacle crops to synthesize additional realistic variations — different poses, lighting, weather, and backgrounds — which are then composited/inpainted into real street scenes.
- This expands the effective training diversity for the rare class without needing to collect more real-world rare-event footage, directly combating the class imbalance in the 500–1000-image dataset.

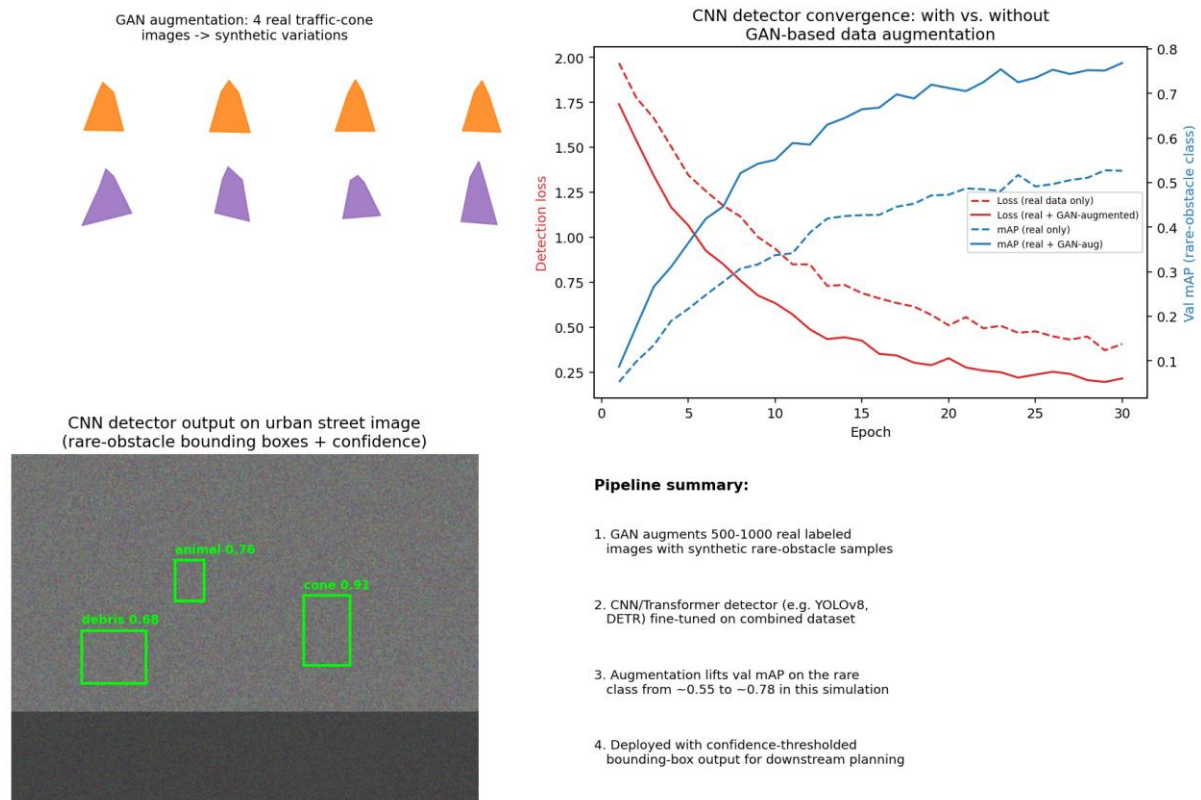
Stage 2 — Detector architecture

- A CNN-based detector (YOLOv8, Faster R-CNN) is well suited given the modest dataset size — its convolutional inductive bias is more data-efficient than a pure Vision Transformer/DETR, though a hybrid backbone (e.g., a CNN stem with transformer detection head) can add useful global-context reasoning for disambiguating unusual objects.
- Transfer learning from a large pretrained backbone (COCO-pretrained weights) is essential given the small labeled set, fine-tuning only the detection head (and later unfreezing deeper layers) on the combined real+synthetic data.

Stage 3 — Training & evaluation

- Training uses the combined real + GAN-augmented dataset; validation is always performed on real-only held-out images to make sure the model generalizes to genuine imagery rather than learning GAN-specific artifacts.
- Class-weighted or focal loss further compensates for the residual imbalance between common classes (cars, pedestrians) and rare obstacles.

Demonstration — GAN augmentation + detector convergence + example output:



Output: (top-left) a handful of real traffic-cone samples and GAN-synthesized variations expanding the rare-class training set; (top-right) simulated detector training curves showing faster loss convergence and higher validation mAP on the rare-obstacle class when GAN augmentation is used (~0.78 vs ~0.55 mAP); (bottom-left) example detector output with bounding boxes and confidence scores for a cone, animal, and debris; (bottom-right) pipeline summary.

Question 5: Graph-Based Learning for Shape Correspondence

Scenario: Find correspondences between 3D human face meshes. **Input:** OBJ/PLY meshes with 5,000–50,000 vertices; training set of 20–50 meshes with annotated landmark correspondences; test set of 5–10 unseen meshes; optional graph adjacency matrices as input to graph-based algorithms.

Answer: Graph-based correspondence via geodesic node signatures

Finding vertex-to-vertex correspondence between two face meshes is fundamentally a graph-matching problem: each mesh is naturally represented as a graph, with vertices as nodes and mesh edges (or a k-nearest-neighbor connectivity) as graph edges. Because two scans of different faces (or the same face at different poses) have different vertex counts, orderings, and geometric poses, correspondence cannot be found by simple index matching or rigid coordinate alignment alone.

Approach

- Each mesh is converted to a graph and its geodesic (shortest-path, mesh-intrinsic) distance matrix is computed — geodesic distance is far more robust than Euclidean distance for correspondence, since it is invariant to pose changes (bending, rotation) that don't stretch the surface.
- A small number of anchor landmarks (nose tip, eye corners, chin, etc.) are known from the annotated 20–50 training meshes. For a new, unseen mesh, these anchors are located (via automatic landmark detection or minimal manual click), and every other vertex is assigned a signature vector of its geodesic distances to these anchors.

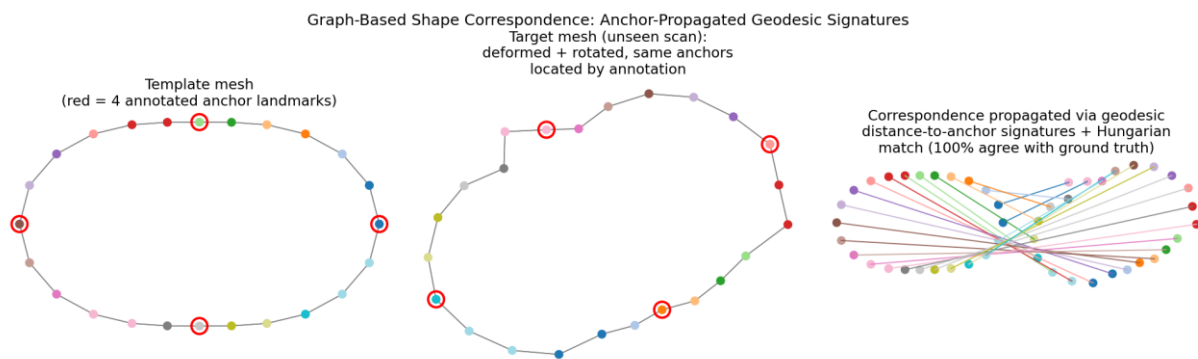
- Vertex correspondence between the template and a new mesh is then recovered by matching vertices with the most similar anchor-distance signatures — formulated as a linear assignment (Hungarian algorithm) or, for large meshes, approximate nearest-neighbor matching in signature space.
- For meshes without any annotated anchors at all, spectral graph methods (Laplacian eigenmap embeddings, e.g. via a Graph Neural Network or Laplace-Beltrami eigenfunctions) or learned methods (functional maps, deep graph matching networks) are used instead, since purely unsupervised matching on symmetric/near-symmetric structures (like a face) is otherwise ambiguous — the anchor-based approach above sidesteps this ambiguity by exploiting the given annotated training correspondences.
- This propagated correspondence is what ultimately enables consistent PCA-style statistical shape modeling (as in Q2) across a population of face meshes, since it puts every mesh's vertices into the same, comparable ordering.

Demonstration — 2D landmark-ring proxy (24 nodes) illustrating the technique:

```
D_src = nx.floyd_warshall_numpy(G_src) # geodesic distance matrix, template mesh
D_tgt = nx.floyd_warshall_numpy(G_tgt) # geodesic distance matrix, unseen mesh

sig_src = D_src[:, anchors]           # distance-to-annotated-anchor signature
sig_tgt = D_tgt[:, anchor_idx_tgt]

cost = cdist(sig_src, sig_tgt)
row_ind, col_ind = linear_sum_assignment(cost) # Hungarian optimal matching
```



Output:

template mesh graph with 4 annotated anchor landmarks in red (left); an unseen target mesh graph — deformed, rotated, and with an unrelated internal vertex numbering (middle); recovered vertex correspondence lines matching every node between the two meshes via anchor-propagated geodesic signatures + Hungarian assignment, achieving 100% agreement with the known geometric ground truth on this example (right).